

DATA ENGINEERING COURSE

Data Cleaning Fundamentals with Pandas

Handling Missing Values, Duplicates, Renaming, Data Types & String Operations

CLASS NOTES

INSTRUCTOR

Tanishq Tyagi

DOCUMENT TYPE

**Complete Study Guide & Interview
Preparation**

Table of Contents

What This Session Covers	3
Session Learning Objectives	4
Why Data Cleaning Matters	5
Handling Missing Values	7
Removing Duplicates	9
Renaming Columns	11
Changing Data Types	12
String Operations	14
Real-World Data Engineering Perspective	16
Revision Summary	18
Quick Revision Sheet	19
Beginner Level	20
Intermediate Level	22
Data Engineer Interview Preparation	24

What This Session Covers

Raw data collected from forms, APIs, databases, and third-party sources is almost never analysis-ready. It arrives with missing entries, duplicated records, inconsistent text formatting, and incorrect data types. This session introduces **data cleaning** — the foundational step of every data engineering and data analysis workflow — using the Python **pandas** library.

Across this session, we worked through six core areas: understanding why cleaning matters, detecting and handling missing values, removing duplicate records, renaming columns for consistency, converting columns to correct data types, and applying string operations to standardize text. Each concept was demonstrated using a sample student dataset containing common real-world data quality issues.

WHY THIS MATTERS

Industry studies consistently show that data scientists and data engineers spend 60–80% of their time cleaning and preparing data before any analysis or modeling can begin. Mastering these operations is one of the highest-leverage skills in a data professional's toolkit.

Session Learning Objectives

By the end of this session, students will be able to:

- Explain why data cleaning is a critical first step in any data pipeline and describe common types of "dirty data."
- Detect missing values using `isnull()` and `isna()`, and quantify them per column.
- Apply appropriate strategies to handle missing values, including `dropna()` and `fillna()` with mean/median imputation.
- Identify and remove duplicate rows using `duplicated()` and `drop_duplicates()`.
- Rename individual columns and standardize entire column-naming conventions using `rename()`.
- Convert column data types using `astype()` and understand why correct dtypes matter for downstream processing.
- Clean and standardize text data using pandas string accessor methods such as `.str.strip()`, `.str.title()`, `.str.upper()`, and `.str.lower()`.
- Connect each cleaning technique to its role in real-world ETL/ELT data pipelines.

Why Data Cleaning Matters

Definition

Data cleaning (also called data cleansing or data scrubbing) is the process of detecting and correcting — or removing — inaccurate, incomplete, inconsistent, or improperly formatted data from a dataset before it is used for analysis, reporting, or machine learning.

Detailed Explanation

Data collected from real-world sources is described as **"dirty data"** when it contains problems that prevent it from being used reliably. The five most common categories of dirty data are:

Type of Dirty Data	Description	Example
Missing values	Cells with no recorded value	Age = NaN
Duplicate rows	The same record appears more than once	"Rahul" entered twice
Wrong data types	Numeric data stored as text, or vice versa	Age = "24" (string)
Messy text	Inconsistent casing, extra spaces, typos	" delhi", "DELHI", "Delhi "
Inconsistent formatting	Mixed naming conventions or units	"IT" vs "it" vs "It"

A useful mental model taught in this session is the **data cleaning pipeline**:

CONCEPT

```
Raw Data
-> Handle Missing Values
-> Remove Duplicates
-> Fix Wrong Formats
-> Standardize Text / Columns
-> Clean, Analysis-Ready Data
```

Why It Is Important

- **Accuracy:** Dirty data produces misleading statistics, broken charts, and incorrect model predictions.
- **Reliability:** Downstream reports and dashboards inherit any error present in the source data ("garbage in, garbage out").
- **Consistency:** Standardized data allows correct grouping, filtering, and joining across tables.
- **Trust:** Business stakeholders lose confidence in analytics teams whose numbers are inconsistent.

Real-World Applications & Industry Use Cases

- **E-commerce:** Cleaning duplicate customer records before calculating customer lifetime value.
- **Banking & Finance:** Standardizing transaction data before fraud-detection models are trained.
- **Healthcare:** Correcting inconsistent patient records before analytics.
- **HR Analytics:** Cleaning employee datasets before generating payroll or headcount reports.

KEY TAKEAWAY

Data cleaning is not an optional pre-step — it is a core engineering responsibility. A pipeline that skips cleaning silently produces incorrect results, which are far more dangerous than a pipeline that crashes.

Handling Missing Values

Definition

Missing values occur when no data is stored for a variable in an observation. In pandas, missing values are typically represented as NaN (Not a Number, from NumPy) or None.

Detecting Missing Values

`isnull()` (alias `isna()`) scans every cell in a DataFrame and returns True where a value is missing and False otherwise. Chaining `.sum()` counts missing values per column, since True is treated as 1.

EXAMPLE

```
import pandas as pd
import numpy as np

data = {
    "Student Name ": ["Rahul", "Sneha ", "AMAN", "rahul", None, "Priya"],
    "Age": [24, np.nan, 25, 22, 24, np.nan],
    "Marks": [85, 90, np.nan, 85, 78, 95],
    "City": [" delhi", "MUMBAI", "Delhi ", " delhi", " pune ", "CHENNAI"],
    "Department": ["IT", "HR", "IT", "IT", "Finance", "HR"],
    "Graduate": [True, False, True, True, False, True],
}

df = pd.DataFrame(data)

# Count missing values in every column
print(df.isnull().sum())
```

EXPECTED OUTPUT

```
Student Name    1
Age             2
Marks           1
City            0
Department      0
Graduate        0
dtype: int64
```

Line-by-Line Explanation

- `df.isnull()` produces a same-shaped DataFrame of booleans marking every missing cell.
- `.sum()` adds down each column, giving one missing-value count per column.
- Columns with no missing values (like City) report 0.

Strategy 1 — Removing Missing Values with `dropna()`

`dropna()` deletes rows (or columns) that contain missing values.

SYNTAX

```
df.dropna(axis=0)  # axis=0 -> drop rows containing any NaN
df.dropna(axis=1)  # axis=1 -> drop columns containing any NaN
```

USE WITH CAUTION

Dropping rows is simple but can discard valuable data. It is only appropriate when the proportion of missing values is very small, or when the missing row is otherwise unusable.

Strategy 2 — Filling Missing Values with `fillna()`

Instead of deleting data, `fillna()` replaces missing values with a specified value — a constant, or a statistically meaningful value such as the column mean or median.

MEAN AND MEDIAN IMPUTATION

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Marks"] = df["Marks"].fillna(df["Marks"].median())
```

Method	Best Used For	Notes
<code>fillna(mean)</code>	Numeric columns without extreme outliers	Sensitive to outliers
<code>fillna(median)</code>	Numeric columns with outliers or skew	More robust, represents the middle value
<code>fillna(mode)</code>	Categorical columns	Fills with the most frequent category
<code>fillna(0)</code>	When "missing" logically means zero	Use carefully — 0 is not always valid

Best Practices

- Always run `isnull().sum()` first to understand the scale of the problem.
- Prefer median over mean for skewed numeric columns (e.g., salary data with a few very high earners).
- Document any imputation strategy used, since it affects downstream statistics and models.
- Consider whether "missingness" itself is meaningful.

Common Mistakes

COMMON MISTAKE

Filling missing numeric values with 0 by default. This silently distorts averages, sums, and downstream calculations — always confirm 0 is logically correct.

COMMON MISTAKE

Forgetting to reassign the result: calling `df["Age"].fillna(df["Age"].mean())` without `df["Age"] = ...` does not modify the DataFrame.

KEY TAKEAWAY

There is no single "correct" way to handle missing values — the right strategy depends on how much data is missing, whether it is numeric or categorical, and what the missingness means in context.

Removing Duplicates

Definition

A **duplicate record** is a row in which every value is identical to another row in the same DataFrame.

Detailed Explanation

`df.duplicated()` returns a boolean Series marking `True` for every row that repeats a previous row (first occurrence is `False`). `df.drop_duplicates()` removes those repeated rows, keeping the first occurrence by default.

EXAMPLE

```
# Check whether duplicate rows exist
print(df.duplicated())

# Remove duplicate rows, keeping the first occurrence
df = df.drop_duplicates()
```

EXPECTED OUTPUT

```
0    False
1    False
2    False
3    False
4    False
5    False
dtype: bool
```

Line-by-Line Explanation

- `duplicated()` compares each row to all rows above it.
- By default it considers *all* columns — use `subset=["col"]` to check specific columns only.
- `keep="first"` (default) marks later occurrences as duplicate; `keep="last"` reverses this; `keep=False` marks every occurrence.
- `drop_duplicates()` physically removes the flagged rows.

Best Practices

- Use `subset=[...]` when a "duplicate" should be judged by a business key (e.g., `employee_id`).
- Always inspect how many rows were removed by comparing `len(df)` before and after.
- Reset the index after dropping rows with `.reset_index(drop=True)`.

Common Mistakes

COMMON MISTAKE

Assuming two rows are duplicates when only some columns match. `duplicated()` without a `subset` requires an exact match across every column.

COMMON MISTAKE

Not standardizing text before checking duplicates — "amit" and " amit " will not be detected as duplicates unless cleaned first.

Real-World Applications

- Removing repeated customer sign-ups caused by form re-submissions.
- Cleaning event-tracking logs where the same event was recorded twice due to a retry mechanism.
- De-duplicating records after merging data from multiple source systems.

KEY TAKEAWAY

Duplicate detection should almost always be paired with text standardization — clean the text first, then check for duplicates, otherwise many true duplicates will be missed.

Renaming Columns

Definition

`rename()` changes the labels of columns (or index values) in a DataFrame, typically used to fix inconsistent, unclear, or improperly formatted column names.

Detailed Explanation

EXAMPLE

```
df = df.rename(columns={" Student Name ": "student_name", "Age": "age"})
```

The `columns` parameter accepts a dictionary mapping `{old_name: new_name}`. Only listed columns are renamed — all others remain unchanged. The result must be reassigned back to `df`, or `inplace=True` must be passed.

Standardizing All Column Names at Once

Rather than renaming columns one at a time, a common production pattern is to programmatically clean every column name in a single step:

EXAMPLE

```
df.columns = (
    df.columns
    .str.strip()           # remove leading/trailing spaces
    .str.lower()           # convert to lowercase
    .str.replace(" ", "_") # replace spaces with underscores
)
```

Best Practices

- Use `snake_case` for column names — it is SQL- and Python-friendly.
- Avoid spaces and special characters in column names.
- Keep column names short but descriptive.
- Rename columns as early as possible so all later code refers to clean, consistent names.

Common Mistakes

COMMON MISTAKE

Typing the old column name slightly wrong in the rename dictionary (e.g., missing a leading/trailing space) — the rename silently does nothing for that column, with no error raised.

KEY TAKEAWAY

Column renaming is about creating a stable, predictable "contract" for every downstream step of the pipeline — inconsistent column names are one of the most common causes of broken production code.

Changing Data Types

Definition

`astype()` converts a pandas Series (or entire DataFrame) from its current data type to a specified new data type, such as converting a floating-point column to an integer.

Detailed Explanation

EXAMPLE

```
df["age"] = df["age"].astype(int)
```

After missing values in the age column were filled (Part 2), the column's dtype is still `float64` because `NaN` can only be represented as a float. Once there are no missing values remaining, `astype(int)` safely converts the column to whole numbers.

Common dtypes	Used For
<code>int64</code>	Whole numbers (ages, counts, IDs)
<code>float64</code>	Decimal numbers, or numeric columns containing <code>NaN</code>
<code>object</code>	Text / mixed data (pandas' default "string" type)
<code>bool</code>	True/False flags
<code>datetime64[ns]</code>	Dates and timestamps
<code>category</code>	Low-cardinality repeated text — saves memory

Best Practices

- Always resolve missing values before converting a column to an integer type.
- Use `category` dtype for repeated categorical text to reduce memory usage.
- Verify conversions with `df.dtypes` or `df.info()` after every change.

Common Mistakes

COMMON MISTAKE

Calling `.astype(int)` on a column that still contains `NaN` — this raises `IntCastingNaNError`.

COMMON MISTAKE

Leaving numeric-looking data as `object` (string) type — this silently breaks arithmetic and aggregation operations like `.sum()`.

Real-World Applications

- Converting timestamp strings from a CSV export into proper `datetime64` objects for time-series analysis.

- Reducing memory footprint of large production datasets by converting text columns to category.
- Ensuring join keys across tables share the same dtype — a common cause of failed database joins.

KEY TAKEAWAY

Correct data types are not cosmetic — they determine which operations (math, sorting, joining, date arithmetic) can be performed correctly and efficiently.

String Operations

Definition

Pandas exposes a **string accessor**, `.str`, that applies vectorized text operations to every value in a column simultaneously — without needing a manual loop.

Detailed Explanation

EXAMPLE

```
df["student_name"] = df["student_name"].str.strip() # remove leading/trailing spaces
df["student_name"] = df["student_name"].str.title() # convert to Title Case
```

Method	What It Does	Example
<code>.str.strip()</code>	Removes leading/trailing whitespace	" Rahul " -> "Rahul"
<code>.str.lower()</code>	Converts text to lowercase	"DELHI" -> "delhi"
<code>.str.upper()</code>	Converts text to uppercase	"it" -> "IT"
<code>.str.title()</code>	Capitalizes the first letter of each word	"amit kumar" -> "Amit Kumar"
<code>.str.replace(a,b)</code>	Replaces one substring with another	"Delhi Ncr".str.replace(" ", "_")
<code>.str.contains(x)</code>	Boolean check for substring presence	Filtering rows containing "error"

Line-by-Line Explanation

- The `.str` accessor is required before any string method — calling `strip()` directly on a Series raises an `AttributeError`.
- Methods can be chained, e.g., `.str.strip().str.title()`, to apply multiple cleaning steps in one line.
- Each method returns a new Series, so the result must be reassigned to persist the change.

Best Practices

- Always `.str.strip()` text columns before comparing, grouping, or checking for duplicates.
- Pick one consistent casing convention per column and apply it uniformly.
- Chain string operations in a single readable line rather than multiple reassignments.

Common Mistakes

COMMON MISTAKE

Forgetting the `.str` accessor, causing `AttributeError: 'Series' object has no attribute 'strip'`.

COMMON MISTAKE

Cleaning text casing but forgetting to also strip whitespace (or vice versa) — both issues frequently co-occur in the same column.

KEY TAKEAWAY

Text standardization is often the difference between a dataset that looks clean and one that actually behaves correctly during grouping, filtering, and joining.

Real-World Data Engineering Perspective

Every technique covered in this session maps directly onto standard stages of a production **ETL/ELT (Extract, Transform, Load) pipeline**. Understanding this connection is essential for data engineering roles, where cleaning logic often runs on millions of rows, on a schedule, without human supervision.

Where These Operations Fit in a Pipeline

Pipeline Stage	Relevant Technique	Purpose
Extract	—	Raw data pulled from source systems — usually the dirtiest stage
Transform — Validation	Missing value detection	Flag incomplete records before reporting
Transform — Deduplication	Removing duplicates	Prevent double-counting in metrics
Transform — Schema Normalization	Renaming, changing types	Enforce a consistent, documented schema
Transform — Standardization	String operations	Ensure values match exactly for joins/filters
Load	—	Clean, typed, deduplicated data loaded into a warehouse or lake

Scalability Considerations

- On large datasets, vectorized pandas operations vastly outperform row-by-row Python loops.
- For datasets too large for memory, the same logic is re-implemented in distributed engines such as Apache Spark, or directly in SQL inside the warehouse (e.g., dbt transformations).
- Deduplication on very large tables is often done using a business key with a window function (e.g., SQL `ROW_NUMBER()`) rather than comparing full rows.

Performance Considerations

- Converting text columns to category dtype can reduce memory usage by an order of magnitude.
- Chaining multiple `.str` operations in one pass is more efficient than repeated reassignments.
- Filling missing values with a precomputed constant is faster than recomputing mean/median in a loop.

Common Industry Scenarios

- A nightly pipeline ingests a CRM export with inconsistent department names — string standardization ensures BI dashboards group correctly.
- An API returns optional fields as null — an engineer must decide whether to drop, impute, or flag these records before loading.
- Two source systems are merged, producing duplicate customer records with slightly different formatting — cleaning logic reconciles them into a single source of truth.

KEY TAKEAWAY

In production, data cleaning is rarely a one-time notebook exercise — it is codified into repeatable, tested, scheduled pipeline logic that runs automatically every time new data arrives.

Revision Summary

Important Concepts

- Dirty data generally falls into five categories: missing values, duplicates, wrong data types, messy text, and inconsistent formatting.
- Missing values can be handled by removal (`dropna`) or imputation (`fillna` with mean/median/mode).
- Duplicate rows are detected with `duplicated()` and removed with `drop_duplicates()`.
- Column names should be renamed to a consistent, code-friendly convention (typically `snake_case`).
- Data types must be corrected with `astype()` — missing values must be resolved first for integer conversions.
- Text data is standardized using the `.str` accessor: `strip()`, `lower()`, `upper()`, `title()`, `replace()`.

Key Syntax

Task	Syntax
Count missing values per column	<code>df.isnull().sum()</code>
Drop rows with missing values	<code>df.dropna(axis=0)</code>
Fill with mean	<code>df["col"].fillna(df["col"].mean())</code>
Check duplicate rows	<code>df.duplicated()</code>
Remove duplicate rows	<code>df.drop_duplicates()</code>
Rename columns	<code>df.rename(columns={"old": "new"})</code>
Convert data type	<code>df["col"].astype(int)</code>
Strip whitespace	<code>df["col"].str.strip()</code>
Title case	<code>df["col"].str.title()</code>

Frequently Misunderstood Concepts

- **Pandas operations are not in-place by default.** Most methods return a new object and must be reassigned unless `inplace=True` is passed.
- **NaN forces float dtype.** A numeric column containing even one missing value cannot be an integer column until the missing value is resolved.
- **`duplicated()` checks exact matches across all columns by default** — near-duplicates with different casing or whitespace are not automatically detected.

Quick Revision Sheet

Core Concepts in One Line Each

- **Data Cleaning:** Fixing missing, duplicate, mistyped, or inconsistently formatted data.
- **Missing Values:** Detect with `isnull()`, resolve with `dropna()` or `fillna()`.
- **Duplicates:** Detect with `duplicated()`, remove with `drop_duplicates()`.
- **Renaming:** Use `rename(columns={...})` for specific columns, or vectorized `.str` chains for all columns at once.
- **Data Types:** Use `astype()`; resolve NaN before converting to int.
- **String Ops:** Always go through the `.str` accessor: `strip` / `lower` / `upper` / `title` / `replace`.

Syntax Cheat Sheet

`df.isnull().sum()` — count missing values per column

`df.dropna(axis=0)` — drop rows with any missing value

`df["c"].fillna(df["c"].mean())` — impute with mean

`df["c"].fillna(df["c"].median())` — impute with median

`df.duplicated()` — flag duplicate rows

`df.drop_duplicates()` — remove duplicate rows

`df.rename(columns={"old":"new"})` — rename specific columns

`df.columns.str.strip().str.lower()` — standardize all column names

`df["c"].astype(int)` — change data type

`df["c"].str.strip().str.title()` — clean and case text

Important Reminders

- Always reassign the result of a pandas operation, or use `inplace=True`.
- Handle missing values before converting a column to an integer type.
- Standardize text (case + whitespace) before checking for duplicates.
- Verify your work with `df.info()`, `df.dtypes`, and `df.head()` after each cleaning step.

Beginner Level

1. What is data cleaning, and why is it necessary?

Data cleaning is the process of identifying and correcting inaccurate, incomplete, duplicate, or inconsistently formatted data. It is necessary because raw data almost always contains quality issues that would otherwise produce misleading analysis, broken reports, or poor model performance.

2. What are the common types of dirty data?

Missing values, duplicate rows, incorrect data types, messy or inconsistently formatted text, and inconsistent naming/formatting conventions across records.

3. How do you check for missing values in a pandas DataFrame?

Use `df.isnull()` to get a boolean mask of missing cells, and chain `.sum()` to count missing values per column, e.g., `df.isnull().sum()`.

4. What is the difference between `dropna()` and `fillna()`?

`dropna()` removes rows or columns containing missing values entirely, while `fillna()` replaces missing values with a specified value (a constant, or a computed statistic like the mean or median), preserving the row.

5. When would you use the mean vs. the median to fill missing values?

Use the mean for roughly symmetric numeric data without significant outliers. Use the median when the data is skewed or contains outliers, since the median is not pulled by extreme values.

6. How do you detect and remove duplicate rows?

`df.duplicated()` returns a boolean Series flagging repeated rows, and `df.drop_duplicates()` removes them, keeping the first occurrence by default.

7. How do you rename a column in pandas?

Use `df.rename(columns={"old_name": "new_name"})`. The result must be reassigned to `df`, or `inplace=True` must be passed.

8. How do you change the data type of a column?

Use `df["column"].astype(new_type)`, for example `df["age"] = df["age"].astype(int)`. Missing values must be resolved first when converting to integer.

9. What does the `.str` accessor do in pandas?

It exposes vectorized string methods (like `.strip()`, `.lower()`, `.title()`) that can be applied to every value in a text column at once, without writing a manual loop.

10. Why does `df["Age"].fillna(0)` sometimes not change the DataFrame?

Because pandas methods return a new object by default rather than modifying in place. Unless the result is reassigned (`df["Age"] = df["Age"].fillna(0)`) or `inplace=True` is used, the original DataFrame is left unchanged.

11. What is the difference between NaN and None in pandas?

None is a native Python object representing absence of value, while NaN (Not a Number) is a special floating-point value from NumPy. Pandas often converts None to NaN internally in numeric columns for consistency.

12. Why might text values like "Delhi" and " delhi " be treated as different categories?

Because pandas compares strings exactly, including case and whitespace. Without applying `.str.strip()` and consistent casing (e.g., `.str.title()`), visually similar values are treated as distinct categories.

Intermediate Level

1. How would you handle missing values differently for numeric vs. categorical columns?

For numeric columns, use mean or median imputation (or a model-based estimate for more advanced cases). For categorical columns, use the mode (most frequent category) or a placeholder like "Unknown," since mean/median have no meaning for text categories.

2. What is the risk of always dropping rows with missing values in a production pipeline?

If missingness is not random, dropping rows introduces bias by systematically removing certain kinds of records. It can also silently shrink the dataset over time as upstream data quality degrades, without triggering an obvious error.

3. How do you check for duplicates based on only a subset of columns?

Pass the relevant columns to the subset parameter: `df.duplicated(subset=["employee_id"])`, which flags rows as duplicate based only on matching values in the specified columns rather than the entire row.

4. Why must missing values be resolved before converting a column to int?

Because NumPy's integer type cannot represent NaN — only floating-point types can. Calling `.astype(int)` on a column still containing NaN raises `IntCastingNaNError`, so imputation or removal must happen first.

5. What is the advantage of the category dtype over object for text columns?

The category dtype stores each unique value once internally and represents rows as integer codes, significantly reducing memory usage and speeding up group-by operations on columns with a small number of repeated values, such as department or status codes.

6. How would you standardize inconsistent column names across an entire DataFrame in one step?

Chain vectorized string operations on `df.columns`, e.g., `df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")`, which trims, lowercases, and replaces spaces across all column labels at once.

7. What is the difference between `keep="first"`, `keep="last"`, and `keep=False` in `drop_duplicates()`?

`keep="first"` (default) retains the first occurrence and drops later duplicates. `keep="last"` retains the last occurrence instead. `keep=False` drops every occurrence of any duplicated row, leaving only unique rows.

8. How would you handle a numeric column stored as strings with inconsistent formatting (e.g., "\$50,000", "45000")?

Use `.str.replace()` to strip non-numeric characters such as currency symbols and commas, then apply `.astype(float)` or `.astype(int)` to convert the cleaned strings into a proper numeric type.

9. Why is vectorized string cleaning (via `.str`) preferred over a Python for loop?

Vectorized pandas operations are implemented in optimized C code under the hood and process an entire column at once, making them significantly faster than iterating row-by-row in native Python.

10. How would you decide the order of cleaning operations in a real pipeline?

A sensible order is: standardize text and column names first, then detect and handle missing values, then remove duplicates (since text cleaning often reveals additional duplicates), and finally correct data types once all values are complete and consistent.

IMPORTANT SECTION

Data Engineer Interview Preparation

Focused on production systems, pipelines, and real-world data engineering scenarios.

1. In a scheduled ETL pipeline, where should data cleaning logic live, and why?

Cleaning logic typically lives in the "Transform" stage — after raw extraction and before loading into the warehouse. This keeps raw source data immutable and auditable while ensuring only clean, standardized data reaches downstream consumers such as BI tools.

2. How would you design a pipeline to handle a source system that occasionally sends duplicate records due to retries?

Deduplicate based on a stable business key combined with an ingestion timestamp, using logic like SQL's `ROW_NUMBER() OVER (PARTITION BY key ORDER BY timestamp DESC)` to keep only the latest version of each record, rather than comparing entire rows.

3. How do you handle missing values in a pipeline that must run unattended on a schedule?

Define an explicit, documented imputation policy per column (e.g., median for numeric, "Unknown" for categorical) so the pipeline behaves deterministically, and log the count of imputed/dropped records on every run so data quality regressions are visible.

4. What is schema drift, and how does column renaming/type standardization help prevent it?

Schema drift occurs when a source system changes column names, types, or structure over time without notice, silently breaking downstream jobs. Enforcing a standardized schema in the transform layer creates a stable contract that isolates downstream consumers from upstream changes.

5. Why is it dangerous to silently drop rows with missing values in production without monitoring?

Silent drops can mask an upstream data quality issue and cause metrics to gradually become incomplete without any alert firing. Production pipelines should log and monitor drop/fill counts so sudden spikes trigger investigation rather than passing unnoticed.

6. How would you clean and standardize data at scale if it no longer fits in memory?

Use a distributed processing engine such as Apache Spark, which applies the same conceptual operations across a cluster of machines in parallel, or push equivalent transformation logic into SQL within the data warehouse (e.g., via dbt).

7. What is the difference between handling data quality in ETL vs. ELT architectures?

In ETL, data is cleaned and transformed before loading, so only clean data lands in the warehouse. In ELT, raw data is loaded first and cleaning happens afterward inside the warehouse, which preserves raw data for reprocessing but requires clear separation between raw and cleaned layers.

8. How would you validate that a cleaning step in your pipeline is working correctly over time?

Add automated data quality checks (e.g., using a framework like Great Expectations or custom assertions) that verify expectations such as "no nulls in required columns" or "no duplicate primary keys" after each run, alerting the team if any check fails.

9. Why might a data engineer choose to keep both raw and cleaned versions of a dataset?

Keeping raw data preserves the ability to reprocess history if a cleaning rule changes, supports auditing/compliance requirements, and allows debugging by comparing raw vs. cleaned values when downstream numbers look incorrect.

10. What performance issue can arise from repeatedly calling `.fillna()` or `.astype()` inside a loop over columns?

Looping in native Python loses the performance benefit of pandas' vectorized, C-backed operations. It is more efficient to apply operations across multiple columns at once, e.g., using `df.fillna({...})` with a per-column dictionary, rather than iterating manually.

11. How would you handle inconsistent categorical values coming from multiple source systems (e.g., "IT", "I.T.", "Information Technology")?

Build and maintain a standardization/mapping table that maps every known variant to one canonical value, and apply it via a join or a `.replace()/map()` operation during transformation, rather than relying only on generic case/whitespace cleaning.

12. What role does data cleaning play before loading data into a dimensional data warehouse model?

Dimension tables require clean, deduplicated, consistently formatted attribute values because they are used for grouping and filtering across the warehouse; dirty dimension data causes fact table joins to fragment a single real-world entity into multiple inconsistent rows.

13. How would you handle a column with mixed data types coming from a poorly governed source?

First profile the column to understand the mix (e.g., using `value_counts()`), then apply a coercion strategy such as `pd.to_numeric(col, errors="coerce")` to convert valid numbers and turn invalid entries into NaN, which can then be handled with the standard missing-value strategy.

14. What trade-off exists between cleaning data early in the pipeline vs. cleaning it at query/reporting time?

Cleaning early ensures every downstream consumer sees the same clean data and avoids duplicated cleaning logic, but requires reprocessing if a rule changes. Cleaning at query time gives flexibility but risks inconsistent results if different analysts apply different logic to the same raw data.

15. How would you explain to a non-technical stakeholder why a dashboard number changed after a "data cleaning" update?

Explain in plain terms what changed (e.g., "duplicate sign-ups were being counted twice, and we removed the duplicates"), quantify the impact, and note that the new number is more accurate — transparency about what changed and why builds trust in the data.

16. What is idempotency in the context of a data cleaning pipeline, and why does it matter?

An idempotent pipeline produces the same clean result no matter how many times it is re-run on the same raw input. This matters because pipelines are frequently re-run (backfills, retries), and non-idempotent cleaning logic can silently corrupt data over multiple runs.